МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РЕСПУБЛИКИ КАЗАХСТАН
СЕВЕРО-КАЗАХСТАНСКИЙ УНИВЕРСИТЕТ ИМ. М. КОЗЫБАЕВА
ФАКУЛЬТЕТ ИНЖЕНЕРИИ И ЦИФРОВЫХ ТЕХНОЛОГИЙ
КАФЕДРА ИНФОРМАЦИОННО-КОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

ОТЧЁТ ПО ООП
МОДУЛЬ 5
Тема «Портфолио веб-разработчика»

Выполнили: студенты группы ИСУ(kz-us)-24-1
Головин Д. П.
Дедов А. А.

Проверила: Астапенко Н. В.

Петропавловск, 2026

Улучшение приложения портфолио веб-разработчика

Введение

Актуальность. Начинающему веб-разработчику важно хранить учебные и реальные работы в одном каталоге, быстро находить нужные кейсы и не терять их при перезапуске приложения. Для этой задачи создан локальный менеджер портфолио с поиском, фильтрами и переносимыми форматами данных.

Цель проекта. Доработать настольное приложение «Портфолио веб-разработчика», добавив надёжное хранение, поиск и фильтрацию записей, проверку ввода и обмен данными.

Задачи проекта: 1) реализовать сохранение и чтение внешних данных; 2) добавить поиск по текстовым и числовым атрибутам; 3) реализовать фильтры и сортировку; 4) проверять ввод и подтверждать удаление; 5) обеспечить перенос данных через JSON и CSV; 6) подготовить диаграммы, блок-схемы и доказательства работы.

1 Реализация хранения

Основное хранилище настольной версии — локальная база SQLite portfolio_data.sqlite3 рядом с программой. Таблицы metadata и projects хранят сведения о портфолио и записях; проект сохраняется полным JSON-payload, а атрибуты для поиска — отдельными столбцами. Запись выполняется транзакционно, версия схемы проверяется при открытии. Старый portfolio_data.json автоматически переносится при первом запуске без SQLite и остаётся резервной копией. JSON применяется для импорта и экспорта, CSV — для обмена таблицей, Markdown — для публикационного текста.

История развития проекта. В отчёте модуля 4 зафиксирован более ранний этап: настольная версия хранила данные в JSON. В модуле 5 SQLite стала основным хранилищем, а исходный JSON сохранился как резервная копия и формат обмена. После учебного прототипа добавлена браузерная PWA-компаньон: она хранит данные отдельно в IndexedDB и переносит их через JSON; автоматической синхронизации между устройствами нет. Офлайн-кэш Service Worker доступен после первого открытия только в защищённом контексте (HTTPS или localhost); при обычном HTTP-доступе по локальной сети он недоступен [9].

2 Поиск и фильтры

Поиск без учёта регистра охватывает название, описание, тип, статус, дату и число страниц, роль, результат, демо и репозиторную ссылки, серверный стек, контекст, выбранное решение, этапы, метрики, теги и сведения о технологиях. Пустой запрос возвращает все проекты. Совместно с поиском работают фильтры по типу, статусу, тегу и избранному; сортируемые заголовки таблицы меняют порядок.

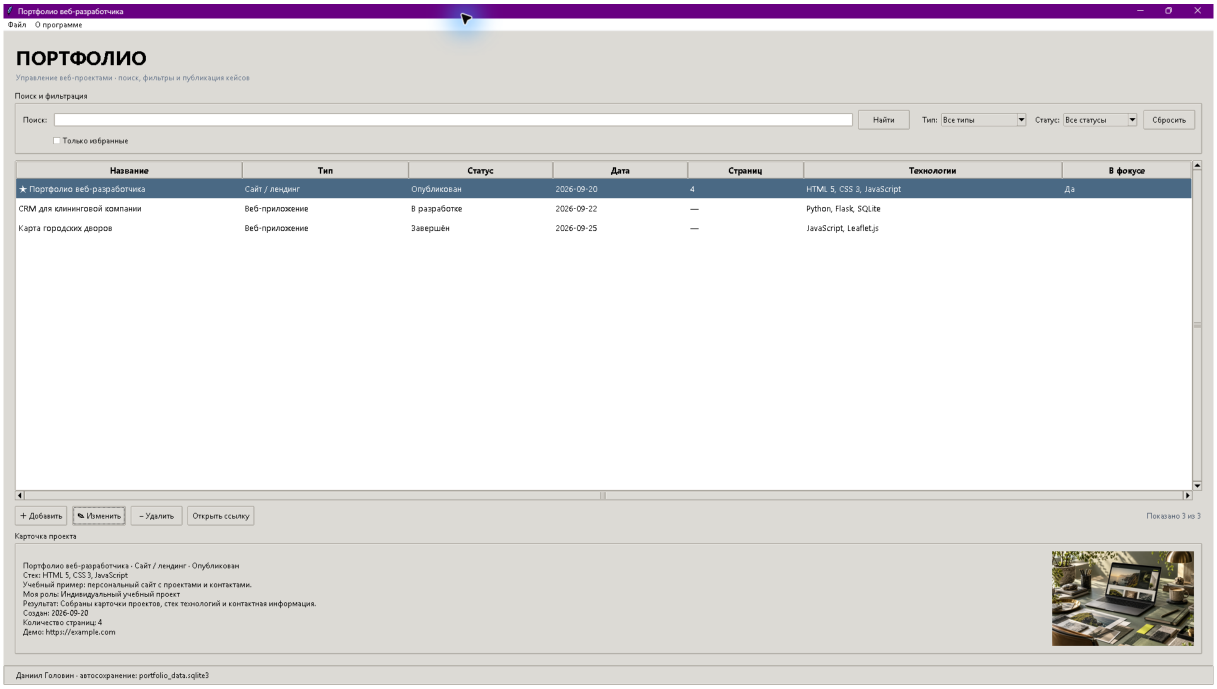
3 Валидация и подтверждения

Название и описание обязательны. Дата должна быть корректной и записанной в формате ГГГГ-ММ-ДД, статус выбирается из заданного набора, URL должен содержать схему HTTP или HTTPS и имя узла. Для сайта количество страниц должно быть положительным целым числом. Ошибочные значения показываются через messagebox; удаление требует явного подтверждения.

4 Структура классов и алгоритмы

В проекте используются Developer, Portfolio, Technology, абстрактный Project и два его подкласса. При старте SQLiteStore проверяет идентификатор приложения, версию и целостность базы; старый JSON проходит полную проверку до замены состояния. Для поиска используется Portfolio.search, после чего применяются фильтры по типу, статусу и избранному и сортировка результатов.

После текстового поиска и фильтрации по типу, статусу, тегу и избранному строки сортируются; интерфейс показывает число найденных записей. SQLite — рабочее хранилище, JSON/CSV — переносимые форматы, Markdown — публикационный экспорт.


Портфолио веб-разработчика
Файл
О программе
ПОРТФОЛИО
Управление веб-проектами · поиск, фильтры и публикация кейсов
Поиск и фильтрация
Поиск:
Найти
☐ Только избранные
Сбросить
+ Добавить
Изменить
– Удалить
Открыть ссылку
Показано 3 из 3
Карточка проекта

Тип: Все типы

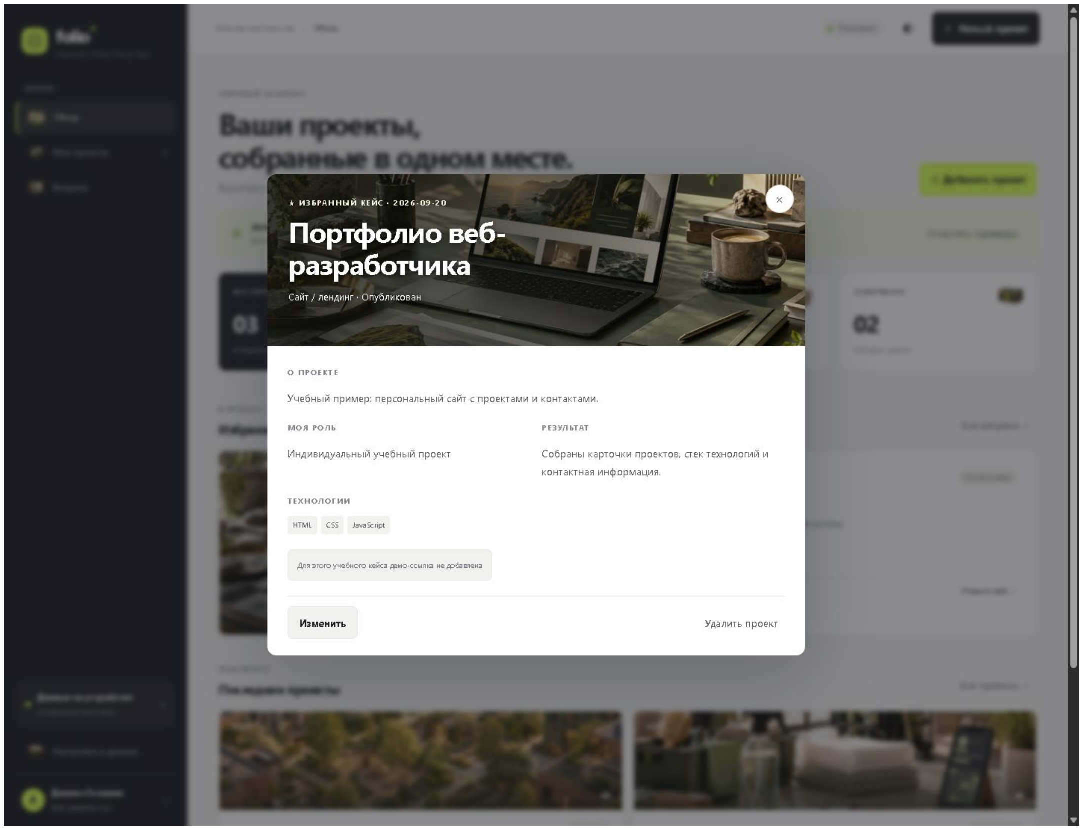
Статус: Все статусы

Название	Тип	Статус	Дата	Страниц	Технологии	В фокусе
★ Портфолио веб-разработчика	Сайт / лендинг	Опубликован	2026-09-20	4	HTML 5, CSS 3, JavaScript	Да
CRM для клининговой компании	Веб-приложение	В разработке	2026-09-22	—	Python, Flask, SQLite	
Карта городских дворов	Веб-приложение	Завершён	2026-09-25	—	JavaScript, Leaflet.js	

Портфолио веб-разработчика · Сайт / лендинг · Опубликован
Стек: HTML 5, CSS 3, JavaScript
Учебный пример: персональный сайт с проектами и контактами.
Моя роль: Индивидуальный учебный проект
Результат: Собраны карточки проектов, стек технологий и контактная информация.
Создан: 2026-09-20
Количество страниц: 4
Демо: https://example.com

Даниил Головин · автосохранение: portfolio_data.sqlite3

Рисунок 1 — Главное окно со списком проектов и предпросмотром обложки



★ ИЗБРАННЫЙ КЕЙС · 2026-09-20

Портфолио веб-разработчика

Сайт / лендинг · Опубликован

О ПРОЕКТЕ

Учебный пример: персональный сайт с проектами и контактами.

МОЯ РОЛЬ

Индивидуальный учебный проект

РЕЗУЛЬТАТ

Собраны карточки проектов, стек технологий и контактная информация.

ТЕХНОЛОГИИ

HTML CSS JavaScript

Для этого учебного кейса демо-ссылка не добавлена

Рисунок 2 — Детальная карточка веб-компаньона, добавленного после учебного этапа

Фрагмент обновления списка по запросу и фильтрам

```
kind, status = self.type_filter.get(), self.status_filter.get()
tag = self.tag_filter.get()
rows = self.portfolio.search(self.query.get())
self.visible = [p for p in rows
    if (kind == "Все типы" or p.project_type == kind)
    and (status == "Все статусы" or p.status == status)
    and (tag == "Все теги" or any(value.casefold() == tag.casefold() for value in p.tags))
    and (not self.featured_only.get() or p.featured)]
self._sort_visible()
self._render_visible(selected_id)
```

Заключение

Проект использует проверяемое SQLite-хранилище и поддерживает миграцию, импорт и экспорт JSON, экспорт CSV/Markdown, расширенный поиск, фильтры, сортировку, валидацию формы и подтверждение удаления. В настольном приложении каждый кейс может иметь обложку и галерею до восьми изображений с alt-текстом, порядком и кадрированием. Веб-компаньон хранит данные отдельно в IndexedDB и переносит их через JSON. Облака, автосинхронизации и публичной публикации нет. Для браузерных записей добавлена атомарная проверка ревизии IndexedDB: если другая вкладка уже сохранила изменения, устаревшая запись отклоняется с понятным сообщением и не заменяет свежую версию.

Актуальный срез на 01.11.2026: настольная версия v86, веб/PWA v64. На коротком desktop-окне 1000×680 интерфейс сохраняет место для списка, а подробности открывает в отдельной прокручиваемой карточке с обложкой. Регрессия сдачного пакета: Python/SQLite — 52/52, web core — 21/21. В изолированном Chromium проверены 12 размеров экрана от 320 до 1440 px на четырёх разделах (48 сочетаний): горизонтального переполнения и ошибок страницы нет; на телефонах видна нижняя навигация, все три обложки загружаются. Проверены поиск, пустое состояние, создание и повторное открытие проекта, офлайн-перезагрузка и JSON-обмен трёх демонстрационных кейсов между версиями. При отказе IndexedDB создание, дублирование, импорт-слияние и замена не меняют сохранённые данные; форма или предпросмотр остаются доступны для повтора. В отдельном аудите доступности 12 сочетаний «раздел × размер» не выявили безымянных контролов, отсутствующих подписей или незагруженных изображений; мобильные цели не меньше 44 px. В трёх размерах окна проверены диалоги и 26 Tab-переходов формы с возвратом фокуса. Контрастный аудит 24 сочетаний desktop/mobile и двух тем ранее показал минимум 5,29:1. Физический телефон и экранный диктор не проверялись. Отдельный двухвкладочный сценарий подтвердил: одновременная инициализация не создаёт аварийное состояние, свежая запись сохраняется, а устаревшая отклоняется; гонка двух атомарных записей даёт ровно одну успешную запись и один конфликт. Ошибок браузера нет.

Состояние проверочной сборки на 30.10.2026. В веб-каталоге доступны карточки и компактный список; подписи сортировки помещаются на экране шириной 320 px. В браузере проверены размеры 1440×900, 390×844 и 320×568, сохранение выбранного вида после перезагрузки и переход Service Worker v47→v48 с сохранением тестовой записи и офлайн-запуском. В настольной версии клавиатурой можно пройти от фильтров к таблице, выбрать проект стрелками и перейти к действиям. Регрессия: 44/44 теста Python/SQLite и 21/21 тест web core. Физический телефон, экранный диктор и клавиатурная проверка нативных диалогов в эту проверку не входили.

Диаграммы и листинги программы

Диаграмма показывает структуру классов и состав карточки. Для блок-схем добавлены пояснения входов, выходов и условий обработки: валидная запись сохраняется; ошибочная не заменяет рабочие данные.

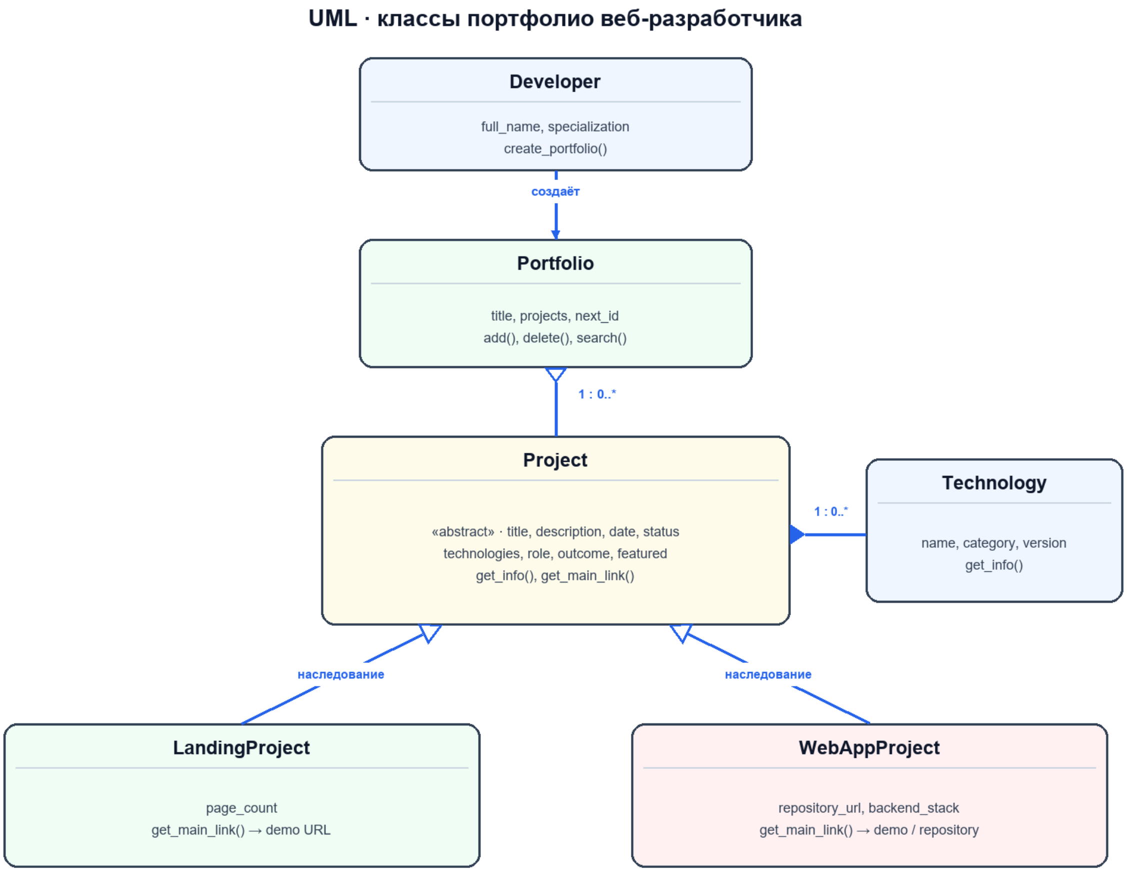
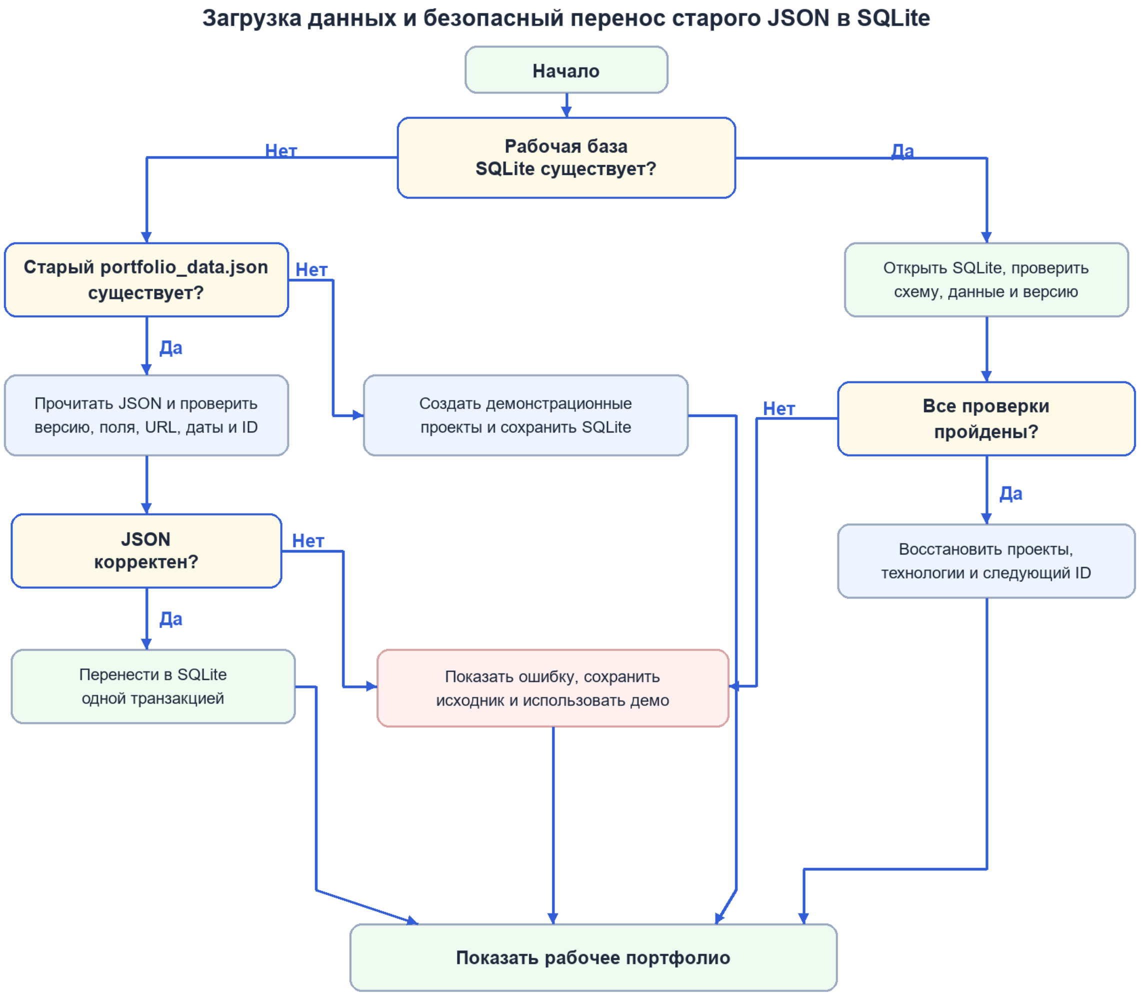

UML · классы портфолио веб-разработчика
Developer
full_name, specialization
create_portfolio()
создаёт
Portfolio
title, projects, next_id
add(), delete(), search()
1 : 0..*
Project
«abstract» · title, description, date, status
technologies, role, outcome, featured
get_info(), get_main_link()
1 : 0..*
Technology
name, category, version
get_info()
наследование
наследование
LandingProject
page_count
get_main_link() → demo URL
WebAppProject
repository_url, backend_stack
get_main_link() → demo / repository

Рисунок 3 — UML-диаграмма классов приложения

Алгоритм открытия и миграции данных SQLite / JSON

Вход: путь к рабочей базе и, при её отсутствии, legacy-файлу JSON. Предусловия: приложение запущено; для JSON проверяются версия схемы, обязательные поля, уникальность ID, даты, статусы, типы и URL. Алгоритм сначала пытается открыть SQLite; если базы нет, валидирует старый JSON и переносит его транзакционно, оставляя исходный файл нетронутым. При отсутствии обоих источников создаёт демонстрационный набор. Выход: проверенный объект Portfolio и число следующего ID. Постусловие: после успешного переноса данные сохранены в SQLite; при ошибке исходный файл и текущее состояние не перезаписываются.



Загрузка данных и безопасный перенос старого JSON в SQLite

Начало
Рабочая база SQLite существует?
Нет
Да
Старый portfolio_data.json существует?
Нет
Открыть SQLite, проверить схему, данные и версию
Да
Прочитать JSON и проверить версию, поля, URL, даты и ID
Создать демонстрационные проекты и сохранить SQLite
Нет
Все проверки пройдены?
Да
JSON корректен?
Нет
Восстановить проекты, технологии и следующий ID
Да
Перенести в SQLite одной транзакцией
Показать ошибку, сохранить исходник и использовать демо
Показать рабочее портфолио

Рисунок 4 — Загрузка, проверка и перенос портфолио между JSON и SQLite

Фрагмент транзакционного сохранения в SQLite

```
connection.execute("BEGIN IMMEDIATE")
_prepare_schema(connection)
connection.execute("DELETE FROM projects")
connection.execute("DELETE FROM metadata")
...
connection.commit()
except sqlite3.DatabaseError as exc:
    if connection is not None:
        connection.rollback()
    raise SQLiteStoreError(f"Не удалось сохранить SQLite-базу: {exc}") from exc
```

Алгоритм добавления и редактирования

Вход — значения полей карточки и, при редактировании, исходный проект. Выход — добавленный или обновлённый объект в рабочем портфолио. Предусловия — форма открыта и тип проекта выбран; постусловие при успехе — список и карточка обновлены, данные сохранены в рабочее хранилище. При ошибке запись не заменяется, форма остаётся открытой и показывает сообщение.

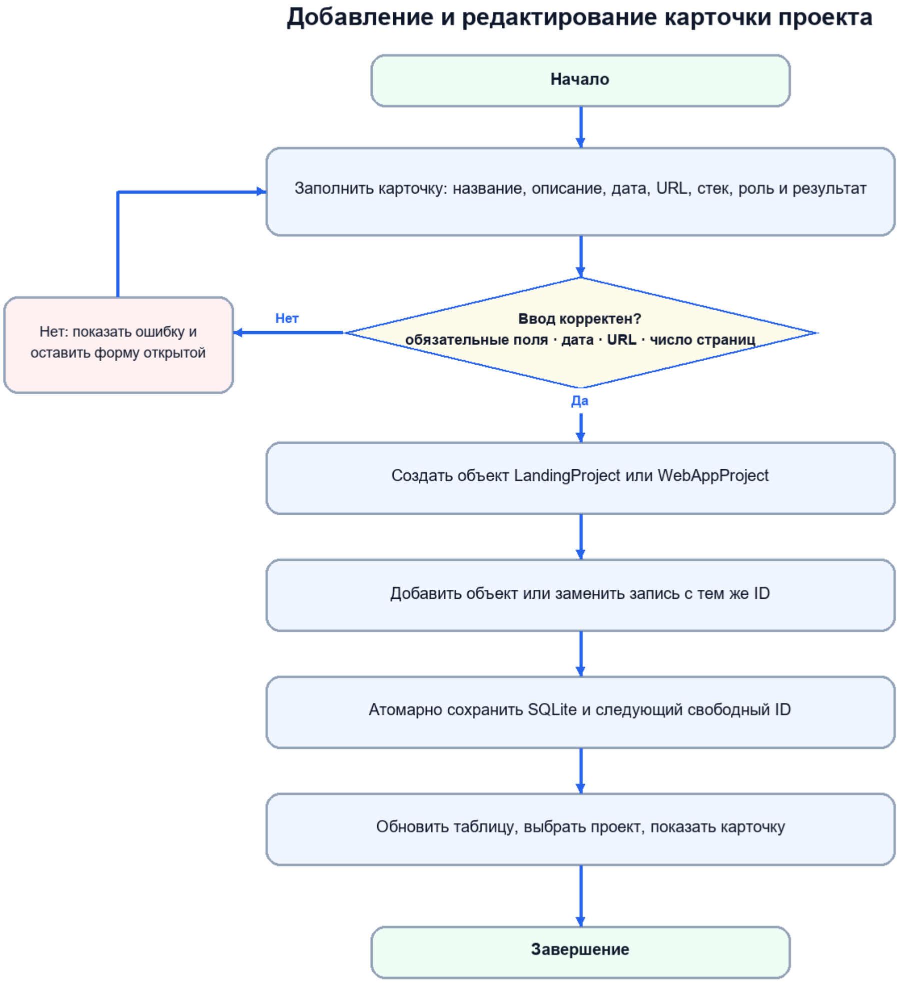

Добавление и редактирование карточки проекта
Начало
Заполнить карточку: название, описание, дата, URL, стек, роль и результат
Ввод корректен?
обязательные поля · дата · URL · число страниц
Нет
Нет: показать ошибку и оставить форму открытой
Да
Создать объект LandingProject или WebAppProject
Добавить объект или заменить запись с тем же ID
Атомарно сохранить SQLite и следующий свободный ID
Обновить таблицу, выбрать проект, показать карточку
Завершение

Рисунок 5 — Проверка формы и транзакционное сохранение карточки в SQLite

Алгоритм поиска и фильтрации

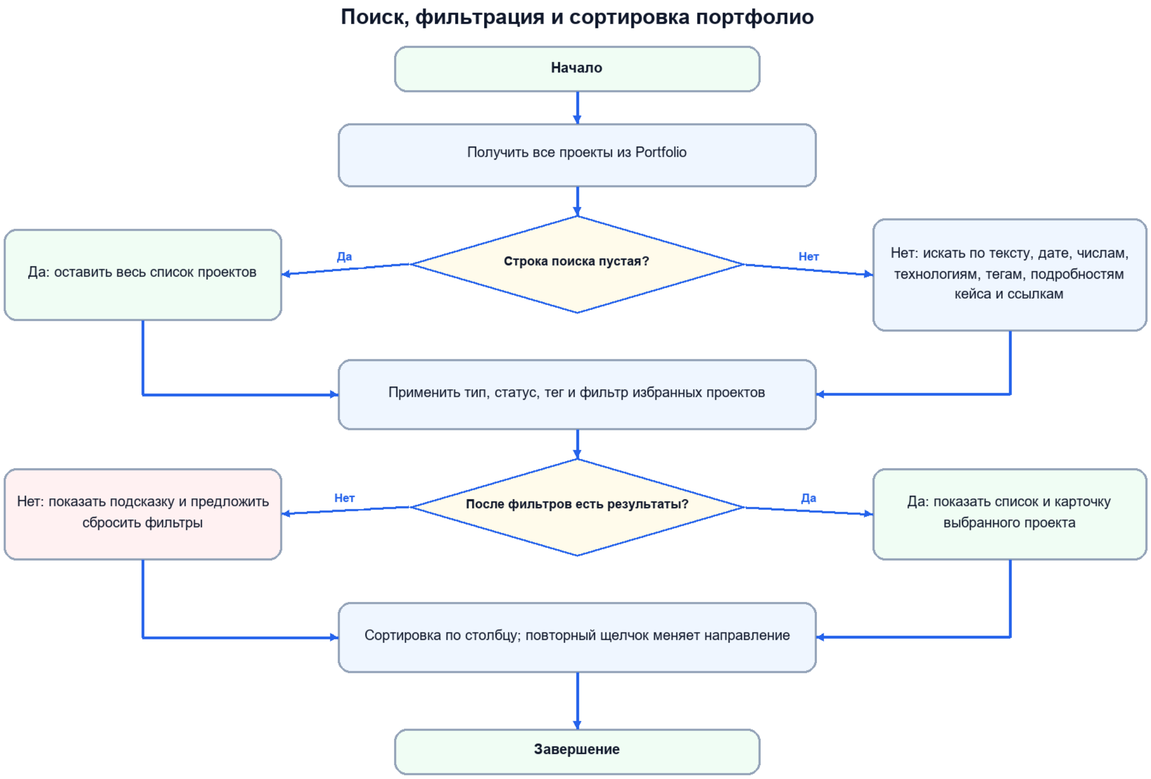

Поиск, фильтрация и сортировка портфолио
Начало
Получить все проекты из Portfolio
Строка поиска пустая?
Да
Да: оставить весь список проектов
Нет
Нет: искать по тексту, дате, числам, технологиям, тегам, подробностям кейса и ссылкам
Применить тип, статус, тег и фильтр избранных проектов
После фильтров есть результаты?
Нет
Нет: показать подсказку и предложить сбросить фильтры
Да
Да: показать список и карточку выбранного проекта
Сортировка по столбцу; повторный щелчок меняет направление
Завершение

Рисунок 6 — Поиск и фильтрация проектов

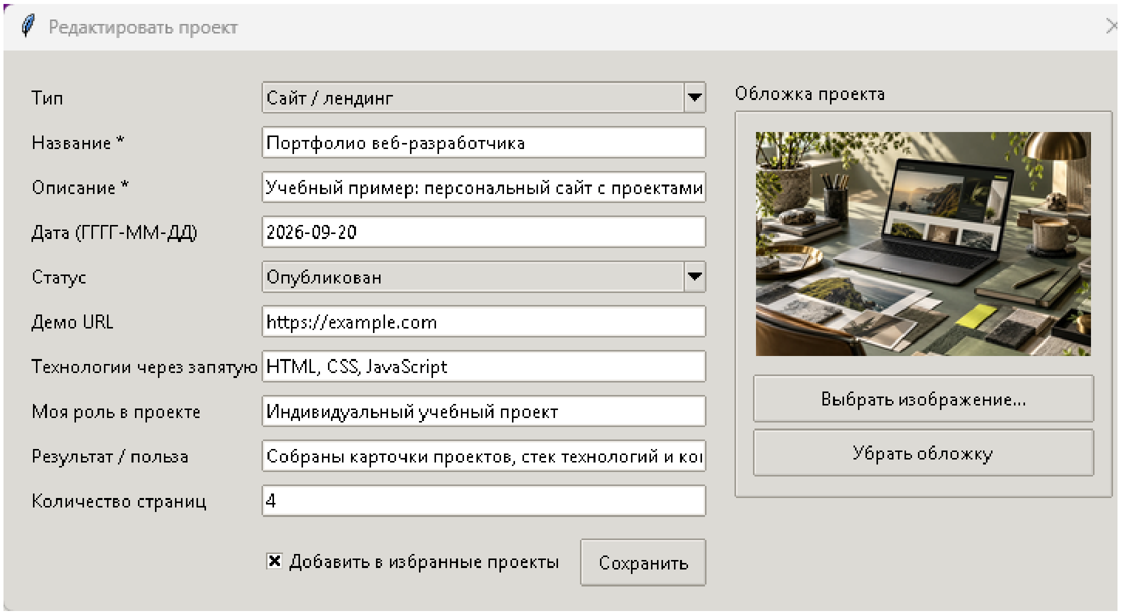
Вход — файл портфолио и выбранные ограничения. Выход — упорядоченный список совпадений и счётчик результатов. Предусловие — текущие карточки загружены и проверены; постусловие — поиск и фильтры не изменяют исходные записи.

Фрагмент поиска по полям портфолио

```
def search(self, query: str) -> list[Project]:
    q = query.strip().casefold()
    if not q:
        return list(self.projects)
    return [p for p in self.projects if q in " ".join(
        [p.title, p.description, p.status, p.project_type, p.created_at,
         p.role, p.outcome, p.demo_url, p.context, p.solution, p.process, p.metrics,
         str(getattr(p, "page_count", "")), *p.tags,
         getattr(p, "repository_url", ""), getattr(p, "backend_stack", ""),
         *(t.get_info() for t in p.technologies)]).casefold()]
```

Форма ввода и фрагменты программы

В форме обязательны название и описание; остальные поля зависят от типа проекта. Общие поля включают дату, статус, демо URL, список технологий, личную роль и результат. Сайт хранит число страниц и может быть отмечен как избранный. Веб-приложение хранит репозиторий и серверный стек. Автоматическая подстановка значений при изменении типа предотвращает потерю введённых данных. Настольная версия позволяет выбрать локальную обложку PNG, JPEG или WebP до 2 МиБ: миниатюра показывается в карточке и форме, изображение хранится вместе с данными проекта [8].



Тип	Сайт / лендинг
Название *	Портфолио веб-разработчика
Описание *	Учебный пример: персональный сайт с проектами
Дата (ГГГГ-ММ-ДД)	2026-09-20
Статус	Опубликован
Демо URL	https://example.com
Технологии через запятую	HTML, CSS, JavaScript
Моя роль в проекте	Индивидуальный учебный проект
Результат / польза	Собраны карточки проектов, стек технологий и ко
Количество страниц	4

☒ Добавить в избранные проекты

Рисунок 7 — Редактирование кейса с предпросмотром и управлением обложкой

Фрагмент проверки данных

```
if not title.strip() or not description.strip():
    raise ValueError("Название и описание обязательны.")
try:
    parsed_date = date.fromisoformat(created_at.strip())
except ValueError as exc:
    raise ValueError("Введите корректную дату в формате ГГГГ-ММ-ДД.") from exc
if parsed_date.isoformat() != created_at.strip():
    raise ValueError("Введите дату строго в формате ГГГГ-ММ-ДД.")
```

Источники

1. Python Software Foundation. tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html (дата обращения: 29.10.2026).
2. Python Software Foundation. csv — CSV File Reading and Writing. https://docs.python.org/3/library/csv.html (дата обращения: 29.10.2026).
3. Python Software Foundation. json — JSON encoder and decoder. https://docs.python.org/3/library/json.html (дата обращения: 29.10.2026).
4. Figma. Portfolio website examples and tips. https://www.figma.com/resource-library/portfolio-website-examples/ (дата обращения: 29.10.2026).
5. GitHub Docs. About your profile. https://docs.github.com/en/account-and-profile/concepts/personal-profile (дата обращения: 29.10.2026).
6. Python Software Foundation. sqlite3 — DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html (дата обращения: 29.10.2026).
7. web.dev. Progressive Web Apps. https://web.dev/learn/pwa/ (дата обращения: 29.10.2026).
8. Pillow (PIL Fork). Image module — opening, loading and creating thumbnails. https://pillow.readthedocs.io/en/stable/reference/Image.html (дата обращения: 29.10.2026).
9. MDN Web Docs. Secure contexts. https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Secure_Contexts (дата обращения: 29.10.2026).